

QuantPawn: A Self-Iteration Trading System

@codingtmd

March 9, 2026

Even starting with the equivalent of an 8-year-old’s intelligence, given sufficient compute resources for iteration, generative AI can eventually beat the best human traders.

Abstract

In the past, quantitative trading heavily relied on experts or statistical mathematicians to analyze the market, discover trading strategies, and apply them for profit. An effective strategy could take years from discovery to practical deployment. And every iteration is heavily depending in smart individual and the understading of the market.

We believe that the advancement of AI, especially the new generation of intelligence represented by Generative AI, fundamentally changes this structure. The inherent reasoning capabilities empower AI with the potential for self-reflection and self-iteration. This article is to present a ai-native structure of quant trading system, which is simple, self-iterationable, and with good-enough extension for future AI direction.

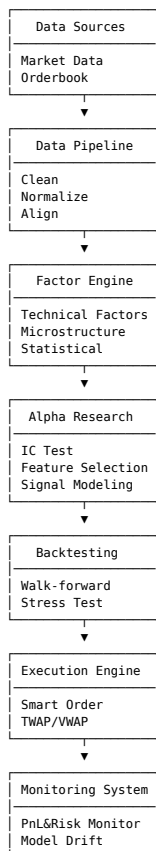
Keywords: AI, Crypto, Quantitative Trading, Multi-Agent Systems

1. Introduction

Modern quantitative trading firms have developed highly structured research pipelines for discovering predictive signals in financial markets. Institutions such as Two Sigma, Citadel, and DE Shaw operate large-scale infrastructures often referred to as alpha factories, where trading signals (alphas) are systematically generated, evaluated, and deployed through a standardized research workflow.

In a typical alpha factory, large volumes of market and alternative data are first collected and processed through data engineering pipelines. Researchers then construct candidate factors—statistical or domain-specific transformations of raw data—which are evaluated using predictive metrics such as information coefficients and return stratification. Promising factors are combined using statistical or machine learning models, including tree-based methods such as XGBoost and LightGBM, or even

advanced algorithms like Neural network, Transformer, etc. Candidate strategies are further validated through backtesting frameworks that simulate realistic trading conditions, after which signals are aggregated through portfolio optimization and executed in live markets. Rather than relying on a single strong model, these systems typically combine large numbers of weak but statistically significant signals to produce stable returns.

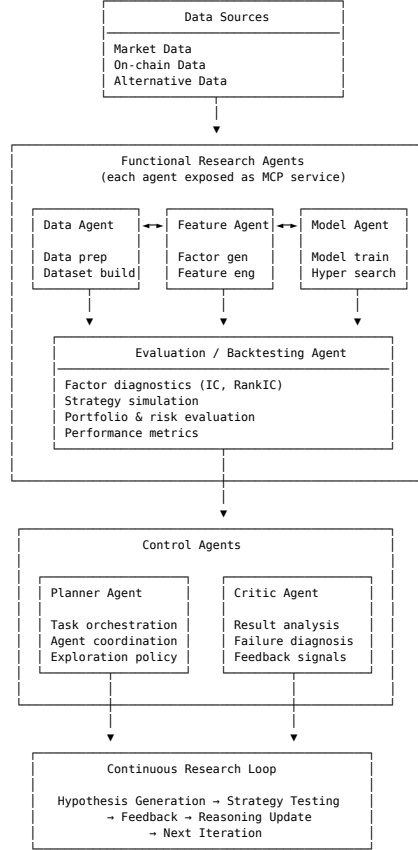


Despite its systematic structure, the alpha discovery process remains largely human-driven. Researchers must manually design candidate factors, select modeling approaches, interpret backtest results, and iteratively refine strategies. As the potential search space of data sources, feature transformations, and model architectures grows exponentially, human-driven exploration becomes increasingly inefficient. This limitation constrains both the scale and speed of signal discovery. Researchers can explore only a small fraction of the possible hypothesis space, and each research cycle—from factor construction to backtesting—may require substantial manual experimentation. In addition, human-driven workflows are susceptible to cognitive biases and methodological inertia, which may limit the diversity of explored strategies. As markets evolve and existing signals decay, the need for continuous strategy discovery further amplifies the burden on research teams.

We propose an AI-native autonomous alpha factory that reconstructs the traditional quantitative research pipeline as a network of interacting AI agents. Rather than

treating tools as external components for human researchers, each research module is implemented as an agent-native service with its own skills, reasoning logic, and MCP-exposed interfaces. In this architecture, agents for data processing, feature construction, model training, factor evaluation, and backtesting collaborate through structured inter-agent calls.

A feedback-driven critic loop continuously analyzes intermediate results and guides subsequent reasoning, enabling the system to iteratively refine hypotheses and trading strategies. By preserving the functional structure of existing alpha factories while replacing human coordination with AI-native orchestration, the framework removes human researchers as the primary bottleneck in large-scale alpha discovery.



2. Search Space

We formulate the whole automation system as a search problem over a large combinational strategy space. A trading signal can be represented as

$$\alpha = f(D, F, M, S, E)$$

where:

- **D:** data sources (market data, on-chain signals, alternative datasets)

- **F**: feature or factor transformations derived from raw data
- **M**: predictive models that map features to expected returns
- **S**: strategy construction rules such as signal aggregation and position sizing
- **E**: execution assumptions including transaction costs, slippage, and market impact

Under this formulation, the strategy search space can be defined as

$$\mathcal{A} = \{f(D, F, M, S, E)\}$$

The objective of alpha discovery is to find the optimal strategy

$$\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \text{Performance}(\alpha)$$

where $\text{Performance}(\alpha)$ represents a performance metric such as Sharpe ratio, risk-adjusted return, or cumulative profit.

Traditional quantitative research explores this space through human-driven experimentation. Rather than treating quantitative modeling as a fixed pipeline, we frame alpha discovery as a large-scale search problem over strategy space, where generative AI agents iteratively explore and refine candidate trading systems.

3. Factor Discovery

In quantitative finance, AI has already been widely applied to factor discovery. Techniques such as genetic programming, symbolic regression, and deep learning models have been used to automatically generate large numbers of candidate factors. These methods significantly expand the search space of potential signals and allow researchers to explore complex nonlinear relationships in financial data. However, a key limitation of traditional approaches is that the search process lacks directional guidance. While AI can generate a vast number of candidate factors, it typically lacks the contextual understanding of the market environment and data characteristics required to steer the search toward promising regions. As a result, the discovery process often relies on human researchers to continuously adjust the search direction, refine evaluation metrics, and interpret intermediate results.

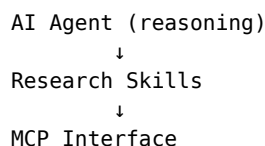
Our approach focuses on replacing this human guidance layer. Instead of relying on researchers to steer the factor search process, AI agents analyze the data, propose directions for factor exploration, and design evaluation criteria. Through iterative feedback and reasoning, the system continuously adjusts its search strategy and corrects its exploration trajectory.

The system evaluates candidate factors in two stages. In the first stage, the factor discovery agent performs lightweight screening based on internally defined evaluation metrics. These metrics may include predictive correlation, statistical stability, and redundancy with existing factors. The purpose of this stage is to rapidly filter out weak or redundant signals while allowing promising candidates to proceed to deeper evaluation.

In the second stage, the surviving factors are passed to the modeling module for full integration testing. Here, the factors are evaluated within the context of the predictive model and the trading strategy. This stage measures the factor’s marginal contribution to overall strategy performance under the Sharpe-oriented evaluation framework described earlier.

4. SKILL + MCP

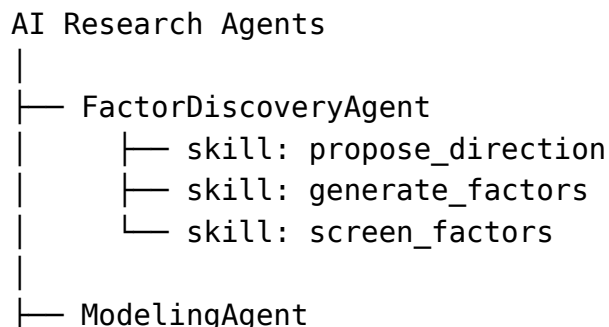
Skills represent research capabilities, while MCP provides the protocol through which AI agents orchestrate these capabilities.



Take factor discovery as an example, it is implemented through a skill-oriented architecture. The system encapsulates the core operations of factor discovery as reusable research skills. These skills represent higher-level research capabilities such as proposing exploration directions, generating candidate factors, screening signals, and evolving the factor search strategy.

Each skill corresponds to a meaningful step in the factor discovery workflow. The skills are then exposed through MCP-compatible interfaces, enabling AI agents to invoke them programmatically during autonomous experimentation.

In this design, factor discovery is no longer implemented as a fixed algorithm, but as a collection of research skills that AI agents can orchestrate to perform autonomous exploration of the factor space.



```

|       |— skill: train_model
|       |— skill: hyperparam_search
|       |— skill: marginal_test
|
└— EvaluationAgent
    |— skill: multi_metric_reasoning

```

5. SKILL v.s. API

This is the key part of how to properly design the skill. As we talked, a skill represents a high-level research capability that encapsulates a complete research action, such as proposing factor discovery directions, generating candidate factors, screening signals, or evaluating strategy performance.

Unlike traditional software APIs, which typically expose low-level computational operations, a skill represents a semantically meaningful research operation. A single skill may internally consist of multiple functions, evaluation steps, and decision logic, but from the perspective of an AI agent it appears as a coherent capability that can be invoked as part of a research workflow.

This distinction is important. In conventional quantitative research systems, APIs usually expose primitive operations such as statistical calculations, feature transformations, or model training utilities. While these primitives are useful for manual programming, they are not well suited for autonomous reasoning systems. AI agents do not naturally reason about sequences of low-level function calls; instead, they operate more effectively when interacting with higher-level capabilities that correspond to meaningful research actions.

For this reason, our system introduces the skill abstraction as the primary interface between AI reasoning and the research infrastructure. Although skills represent higher-level capabilities, we intentionally present their invocation in an API-like pseudocode format. This representation provides several advantages. First, it offers a clear and concise way to document the inputs and outputs of each capability. Second, the structured form aligns naturally with tool-calling mechanisms used by modern AI agents. Finally, the pseudocode format allows readers to understand the semantics of each capability without requiring detailed implementation code.

In this sense, the pseudocode representation should be interpreted as a conceptual interface specification rather than a literal software API. The underlying implementation may involve complex internal logic, state management, or iterative procedures, but these details are abstracted away behind the skill interface.

Example Skill: Candidate Factor Generation(Not Real One)

The following pseudocode illustrates a factor generation skill inside the factor discovery agent.

- ↪ Unlike a simple function that returns raw factor expressions, the skill uses market context,
- ↪ existing factor knowledge, **and** discovery goals to generate structured candidate factors
- ↪ together **with** explanations **and** next-step suggestions.

```

class GenerateCandidateFactorsSkill:

    def __init__(self, factor_library, search_memory, expression_engine):
        self.factor_library = factor_library
        self.search_memory = search_memory
        self.expression_engine = expression_engine

    def run(self, market_context, existing_factor_pool, discovery_goal, num_candidates):
        """
        Generate candidate factors as a structured research action.
        """

        # Step 1: infer promising discovery directions
        directions = self._propose_directions(
            market_context=market_context,
            existing_factor_pool=existing_factor_pool,
            discovery_goal=discovery_goal
        )

        # Step 2: generate candidate expressions under each direction
        raw_candidates = []
        for direction in directions:
            expressions = self.expression_engine.generate(
                theme=direction["theme"],
                data_fields=direction["suggested_data"],
                num_candidates=num_candidates
            )

            for expr in expressions:
                raw_candidates.append({
                    "expression": expr,
                    "theme": direction["theme"],
                    "rationale": direction["rationale"]
                })

        # Step 3: remove invalid or highly redundant candidates
        filtered_candidates = self._filter_candidates(
            raw_candidates,
            existing_factor_pool=existing_factor_pool
        )

        # Step 4: annotate candidates with explanations and family labels
        structured_candidates = []
        for candidate in filtered_candidates:
            structured_candidates.append({
                "factor_id": self._make_factor_id(candidate["expression"]),
                "expression": candidate["expression"],
                "family": self._assign_family(candidate),
                "explanation": self._explain(candidate),
                "expected_signal_type": self._infer_signal_type(candidate)
            })

        # Step 5: update search memory for future iterations
        self.search_memory.record_generation(
            discovery_goal=discovery_goal,
            generated_factors=structured_candidates
        )

```

```

# Step 6: produce next-step recommendation
recommendation = self._suggest_next_step(structured_candidates)

return {
    "candidates": structured_candidates,
    "recommendation": recommendation
}

```

The skill is exposed to AI agents through an MCP-compatible interface, which can be invoked as below.

```

{
  "tool": "generate_candidate_factors",
  "description": "Generate candidate factors under a proposed discovery goal",
  "parameters": {
    "market_context": {
      "type": "object",
      "description": "Summary of current market regime and data availability"
    },
    "existing_factor_pool": {
      "type": "array",
      "description": "Existing factors and their diagnostic summaries"
    },
    "discovery_goal": {
      "type": "string",
      "description": "Target signal type such as momentum, reversal, or volatility prediction"
    },
    "num_candidates": {
      "type": "integer",
      "description": "Number of candidate factors to generate"
    }
  }
}

```

Example response

```

{
  "candidates": [
    {
      "factor_id": "fac_10231",
      "expression": "zscore(funding_rate,20) * delta(price,3)",
      "family": "funding_momentum_interaction",
      "expected_signal_type": "reversal",
      "explanation": "captures divergence between leveraged positioning and short-term price  
↔ momentum"
    },
    {
      "factor_id": "fac_10492",
      "expression": "rolling_mean(open_interest_change,10) / volatility_20d",
      "family": "derivatives_flow",
      "expected_signal_type": "trend_confirmation",
      "explanation": "measures accumulation of leveraged positions relative to volatility"
    }
  ]
}

```



```

"recommendation": "submit top candidates to screening skill for rank-IC and redundancy
↪ evaluation"
}

```

6. Modeling

A key design principle of our system is to fix a stable, controllable, and reproducible modeling backbone while delegating experimental exploration to AI agents. Instead of allowing unrestricted model invention, the modeling layer is grounded in a quantile regression framework implemented with XGBoost. Key modeling operations—including hyperparameter search, factor subset selection, and feature fusion experiments—are exposed through MCP-compatible interfaces. These interfaces allow AI agents to programmatically invoke modeling capabilities and design experiments through structured reasoning and natural-language-driven instructions.

In our work, AI does not replace the modeling foundation itself, but instead autonomously explores the experimental space built on top of it. When a new factor is introduced, the system does not evaluate it in isolation. Instead, the factor is tested in combination with the existing factor set, measuring its marginal contribution under the current modeling framework. The modeling agent launches a series of experiments involving different hyperparameter settings and factor subsets, and evaluates the resulting strategies through backtesting metrics.

Based on the experimental results, the system adaptively refines model configurations and determines whether the new factor improves the overall strategy. The module then produces structured feedback indicating whether the factor should be accepted, rejected, or further refined.

The major objective of the modeling module is to maximize risk-adjusted trading performance:

$$(\theta^*, F^*) = \arg \max_{\theta, F' \subseteq \mathcal{F}} \text{Sharpe}(\text{Model}(F'; \theta))$$

where

- θ denotes model hyperparameters
- F' represents a subset of candidate factors
- $\mathcal{F}$ is the current factor pool
- $\text{Sharpe}(\cdot)$ measures risk-adjusted strategy performance.

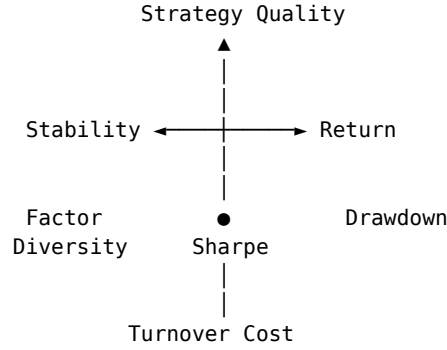
When a new candidate factor f_{new} is proposed, its usefulness is determined by evaluating its marginal improvement to the current factor set:

$$\Delta_{\text{new}} = \text{Sharpe}(F_{\text{base}} \cup \{f_{\text{new}}\}; \theta^*) - \text{Sharpe}(F_{\text{base}}; \theta_0)$$

where F_{base} denotes the existing factor set and θ_0 represents the baseline model configuration. A factor is considered beneficial if it produces a positive and stable improvement in Sharpe ratio under the current modeling framework.

Real-world trading strategies must balance multiple considerations, including profitability, drawdown stability, turnover, and robustness across market regimes. Traditional modeling pipelines typically address this challenge through manually designed multi-objective optimization schemes or rigid scoring functions. However, these approaches often struggle to capture the nuanced trade-offs required for long-term strategy evolution.

In contrast, our system treats Sharpe ratio as the primary anchor metric, while allowing AI agents to reason over a broader set of diagnostic signals produced during modeling and backtesting. These signals include metrics such as cumulative return, drawdown characteristics, turnover, prediction stability, and factor redundancy. Rather than optimizing a fixed scalar objective, the AI agent performs reasoning-driven evaluation, synthesizing these metrics to produce a holistic assessment of candidate strategies. This allows the system to recommend modeling adjustments and factor selections that better align with the long-term evolutionary direction of the strategy quantity, rather than optimizing a single short-term performance indicator.



AI Reasoning-Based Evaluation

7. MCP Interface Example

Let's use a simple example to explain how MCP interfaces work here with agent. Here we use pseudocode to demo the idea. To enable AI agents to autonomously conduct modeling experiments, the capabilities of the modeling module are exposed through MCP-compatible interfaces. Each interface represents a callable research skill that can be invoked programmatically by AI agents during the research process.

The following example illustrates a simplified MCP interface for training the quantile model.

```
{
  "tool": "train_quantile_model",
  "description": "Train XGBoost quantile regression model for factor evaluation",
  "parameters": {
    "factors": {
      "type": "array",
      "description": "List of factors used as model features"
    },
    "hyperparameters": {
      "type": "object",
      "description": "XGBoost model hyperparameters"
    },
    "train_range": {
      "type": "string",
      "description": "Training time range"
    }
  }
}
```

For example, the agent may decide to test the factor in combination with existing signals.

```
train_quantile_model(
  factors = ["momentum_5d", "funding_rate", "new_factor"],
  hyperparameters = {
    "max_depth": 6,
    "eta": 0.05,
    "subsample": 0.8
  },
  train_range = "2022-01-01 to 2023-12-31"
)
```

The modeling system then trains the model and evaluates the resulting trading strategy through backtesting. The result is returned to the AI agent as a structured response containing key evaluation metrics.

```
{
  "sharpe": 1.42,
  "return": 0.23,
  "max_drawdown": 0.11,
  "turnover": 0.34
}
```

These metrics are subsequently analyzed by the AI agent within the Sharpe-anchored multi-metric evaluation framework, allowing the agent to reason about the usefulness of the factor and propose further experiments.

8. Feedback-Driven Loop

A key feature of our system is the bidirectional feedback loop between factor discovery and modeling. While the factor discovery module proposes new candidate factors

and performs preliminary screening, the modeling module provides deeper evaluation results based on full model training and backtesting. The feedback produced by the modeling stage—including improvements or degradations in Sharpe ratio and other metrics—is fed back into the factor discovery agent.

This feedback allows the system to iteratively refine its search strategy. Factors that consistently improve model performance guide the generation of new related candidates, while unsuccessful directions are gradually abandoned. Through repeated cycles of proposal, evaluation, and feedback, the system progressively evolves the factor space toward more robust signals.

In this architecture, factor discovery is no longer a static search process but becomes a self-correcting exploration system, where AI agents continuously adapt their search direction based on observed strategy outcomes. This is why we say, given enough computing power, even each step is minor, it will eventually beats the best trader.

9. Human Participation

While AI systems are capable of large-scale search and experimentation, human intuition and domain knowledge can still help accelerate convergence toward promising research directions and avoid unproductive regions of the search space.

Our system shifts the role of humans from primary decision-makers to cognitive advisors. To enable lightweight human participation, we integrate a communication way based on OpenClaw and Discord. Inside the Discord channels, developers and researchers can discuss observations, propose ideas, or analyze experimental outcomes. The OpenClaw bot monitors these discussions, summarizes key insights, and converts them into structured feedback that can be consumed by the AI system.

Importantly, the system does not depend on human input to function. However, when available, human cognition and imagination can help guide exploration and accelerate the discovery process. Over time, as AI agents become increasingly capable of interpreting data and generating hypotheses independently, the role of human participation may gradually diminish.

10. Application in Cryptocurrency Markets

The system described in this work is applied primarily to cryptocurrency markets. While the proposed architecture is general and can be extended to other asset classes, the crypto market provides a particularly suitable environment for AI-driven autonomous research systems.

First, cryptocurrency markets operate continuously, 24 hours a day and 7 days a week, unlike traditional financial markets that follow fixed trading sessions. This continuous operation allows AI agents to observe market behavior, conduct experiments, and update strategies without interruption, enabling faster research iteration cycles.

Second, crypto markets provide highly accessible and granular data. Market data—including prices, funding rates, order book dynamics, and derivatives information—is widely available through public APIs. The openness of the ecosystem makes it possible to construct large-scale data pipelines and conduct automated experimentation with relatively low barriers.

Third, the crypto market is characterized by rapid regime shifts and evolving market structures. New participants, instruments, and trading behaviors emerge frequently. This dynamic environment creates a natural setting for adaptive systems that continuously generate and evaluate new trading signals.

And the most important, the crypto ecosystem itself is natively digital and programmable, making it highly compatible with automated research and execution infrastructures.

11. Conclusion

A central idea of our approach is to transform the factor discovery and modeling process into a self-iterative AI system.

By redesigning the research pipeline as an AI-native system capable of autonomous iteration, the problem becomes computationally scalable. Under this formulation, quant trading is no longer constrained by human research capacity. Instead, it becomes a compute-driven optimization problem, where improved solutions can be obtained by increasing computational resources and expanding the search process. In this paradigm, computational scalability replaces human effort as the primary limiting factor.

Evolution is nature’s greatest art. In the age of AI, the role of us shifts from designing strategies to building systems that create them.